

GenAI Expert Chatbot

**A Multimodal Retrieval-Augmented Generation System
for the Generative AI Domain**

Team Members

Mostafa Mahmoud Mohamed Noseir

Mohamed Mahmoud Mohamed Noseir

Nourseen Khaled Gouda Ibrahim

Rana Sameh Awad Abdalla

Instructor

Eng. Aya Hisham

DEPI R4 Graduation Project Report

Project Overview

GenAI Expert Chatbot is a locally-deployed, offline Multimodal Retrieval-Augmented Generation (RAG) system built to answer technical questions within a specific domain: Artificial Intelligence, Machine Learning, Deep Learning, and Large Language Models. Rather than relying on a general-purpose language model's memorized knowledge, the system grounds every answer in a private corpus of 380 technical documents, retrieving and reasoning over the *visual content* of the exact pages that support each answer, and citing them explicitly.

The defining architectural choice of this project is that it performs **visual document retrieval rather than traditional text-based RAG**. No OCR or text-extraction step exists anywhere in the pipeline. PDF pages are rendered directly as images, embedded as images, retrieved as images, and read as images by the answering model. This decision was made deliberately, after an earlier prototype using a conventional OCR/chunking pipeline, to avoid the systematic loss of tables, mathematical notation, figures, and layout structure that text-extraction pipelines introduce.

The entire system — embedding model, vector database, and answer-generating vision-language model — runs locally on a single consumer laptop GPU (8GB VRAM), with no dependency on external APIs or cloud services once models are cached to local storage.

Problem Statement and Motivation

General-purpose large language models are broad but shallow with respect to any single specialized domain, and they cannot cite verifiable sources for their claims. A student or researcher working through a large personal collection of papers and technical books faces two related problems: general-purpose chatbots may answer confidently but incorrectly (hallucination), and manually searching hundreds of PDFs for a specific fact, formula, or explanation is slow and error-prone.

Traditional text-based RAG systems address the hallucination problem by grounding answers in retrieved text, but they introduce a different failure mode: OCR and layout-parsing pipelines routinely mangle exactly the content that is hardest to extract and most valuable in technical material: multi-column layouts, nested equations, tables, and figures. This project's visual retrieval approach sidesteps this by never converting the page away from its original, information-complete visual form; PDF pages are rendered directly as images, embedded as images, retrieved as images, and read as images by the answering model, with no OCR or text-extraction step anywhere in the final pipeline.

Main Goal

The goal is to build a production-quality, fully offline Multimodal RAG system that can answer questions about a private technical corpus with accurate, page-level source citations, deployable as a working local chat application.

The system is designed to:

- Search a Generative AI document corpus.
- Retrieve relevant pages or chunks.
- Send retrieved context to a generative model.
- Generate an answer grounded only in the retrieved documents.
- Provide references to the source pages used in the answer.
- Operate entirely within the constraints of consumer-grade hardware (a single 8GB-VRAM GPU).
- Provide a usable local chat interface for interacting with the system.
- Establish a quantitative basis (retrieval evaluation metrics) for assessing system quality, and an honest accounting of the system's current limitations.

Dataset Overview

Category	Count	Format	Primary Source
Research papers	360	PDF	arXiv (open access)
Technical books	20	PDF	See disclosure below
Total documents	380		
Total page images (post-processing)	22,664	PNG	Rendered from source PDFs

Domain coverage: Machine Learning, Deep Learning, Computer Vision, Natural Language Processing, Large Language Models, Retrieval-Augmented Generation, Generative AI (VAEs, GANs, diffusion models), agentic AI systems, mathematics for machine learning, and probabilistic machine learning. Representative book titles include *Deep Learning* (Goodfellow, Bengio), *Natural Language Processing with Transformers*, *Hands-On Large Language Models*, *Mathematics for Machine Learning*, *Building AI Agents with LLMs*, *RAG and Knowledge Graphs*, and *Probabilistic Machine Learning: Advanced Topics*.

Data source disclosure: the 360 research papers were obtained legitimately from arXiv, an open-access repository. The 20 books were **not** obtained through licensed or officially authorized channels, due to access constraints during the project.

Main Components in earlier phase ‘Traditional RAG’:

1. PDF and Markdown Extraction:

The notebooks convert books and research papers into usable formats.

- Book2MD.ipynb: converts books from PDF to Markdown.
- Papers2MD.ipynb: converts research papers from PDF to Markdown.

2. Data Cleaning and Chunking

Cleaning_ & Chunking_Markdown.ipynb cleans extracted Markdown files and splits them into smaller chunks.

The text-based pipeline produced:

- **361 cleaned records**
- **17,590 text chunks**

3. Embedding and Vector Storage

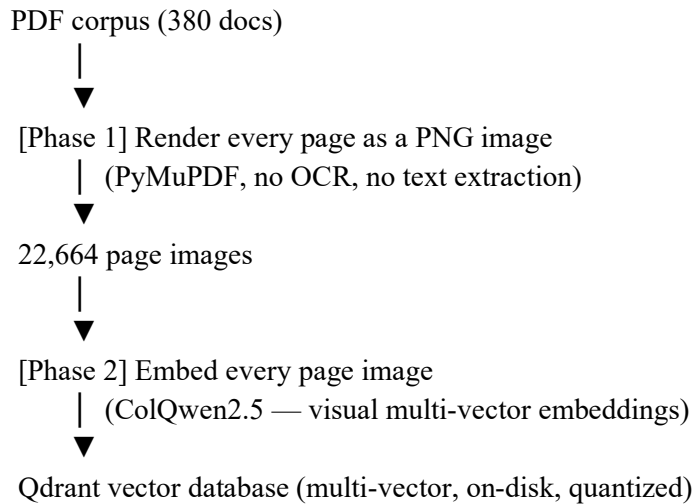
We use to embed Text:

- BAAI/bge-base-en-v1.5
- FAISS vector index
- 17,590 text chunks

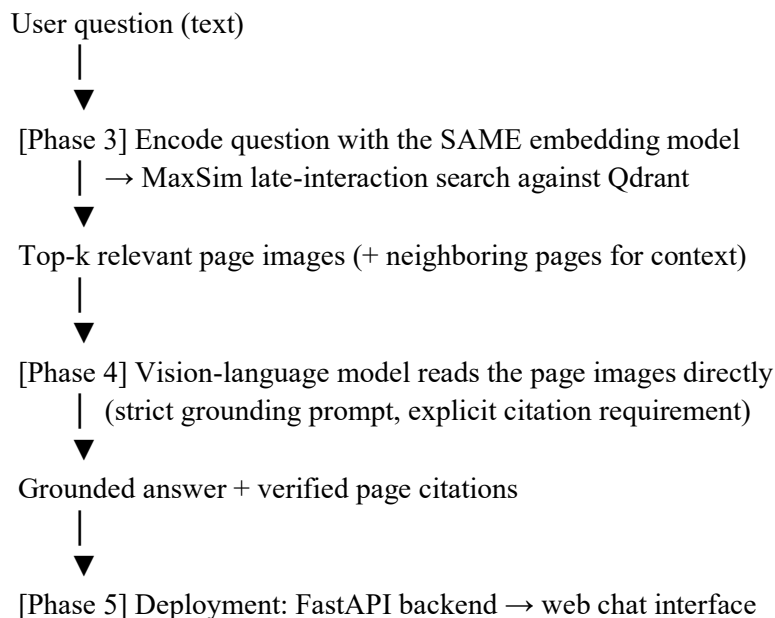
System Architecture

The system is organized as a linear pipeline from raw PDFs to a deployed chat interface, with a strict separation between an offline **indexing pipeline** (run once per corpus, or incrementally as documents are added) and an online **query pipeline** (run per user question).

— INDEXING PIPELINE —

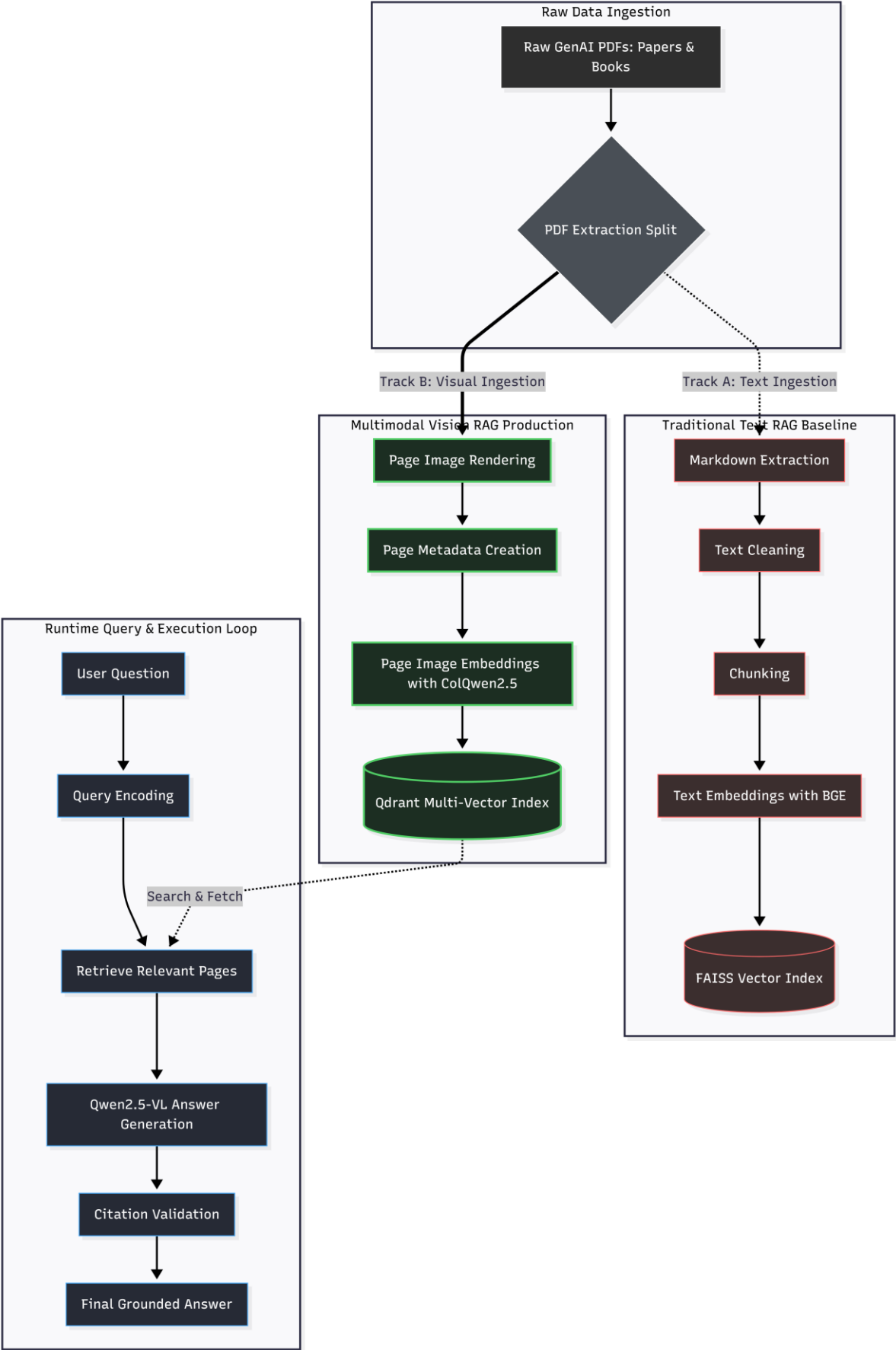


— QUERY PIPELINE —



Why the embedding model appears in both pipelines: ColQwen2.5 is a single model used for two purposes — encoding page images during indexing, and encoding the user’s text query at retrieval time — because both are projected into the same shared vector space, which is what makes similarity search between a text query and image pages meaningful.

Full Project Pipeline Diagram



Technology and Model Stack

Layer	Technology	Rationale
PDF rendering	PyMuPDF (fitz)	Pure Python, no external binary dependency, precise DPI/resolution control
Visual embedding / retrieval model	ColQwen2.5 (vidore/colqwen2.5-v0.2), via colpali-engine	Late-interaction (ColBERT-style) multi-vector visual document retrieval; represents each page as a grid of patch-level embeddings rather than one pooled vector, giving fine-grained matching well suited to text-dense technical pages
Quantization	4-bit NF4 (bitsandbytes)	Required to run the embedding model within an 8GB VRAM budget
Vector database	Qdrant (native server, local)	Production-grade on-disk, memory-mapped multi-vector storage with native MaxSim scoring, essential at the corpus's actual scale
Vision-language model (generation)	Qwen2.5-VL-7B-Instruct , 4-bit NF4 quantized	Strong document, OCR, table, and chart understanding at a size that fits consumer VRAM constraints in 4-bit precision
Backend framework	FastAPI + Uvicorn	Modern, async-capable Python API framework
Frontend	Custom HTML/CSS/JavaScript	Lightweight, dependency-free, fully local chat interface
Runtime environment	Windows, conda environment, fully offline	No cloud API dependency after initial model download

Implementation — Phase by Phase

Phase 0: Traditional Text-Based RAG Baseline

The initial phase of the project established a traditional text-based Retrieval-Augmented Generation pipeline as our baseline architecture. The pipeline executed five core steps:

1. **Markdown Extraction:** Raw PDFs were scraped to extract textual characters into an unformatted markdown layout.
2. **Text Cleaning:** Structural noise, headers, footers, and text-parsing anomalies were filtered out.
3. **Chunking:** Cleaned markdown strings were split into fixed-size semantic overlapping segments.
4. **Text Embeddings:** Chunks were encoded into dense vector representations using a text-based model (BGE).
5. **FAISS Vector Index:** The generated vectors were stored in a local FAISS index for runtime semantic text retrieval.

The Limitation and Strategic Pivot

While this approach proved highly efficient for lightweight research papers, it ultimately failed when scaling to massive, high-volume educational materials. The primary bottleneck was that **extracting heavy academic books into clean Markdown required massive computational resources and extreme processing times** that surpassed our available hardware capabilities.

Furthermore, text-based parsers completely stripped away visual tables, architectural flowcharts, mathematical notation, and spatial layouts essential to answering complex technical queries. Because of these severe compute constraints and layout structural losses, we abandoned the text-only strategy and shifted to a natively visual, highly scalable **Multimodal RAG Strategy**.

Phase 1 — PDF to Page Images

Every PDF is opened with PyMuPDF and rendered page-by-page to PNG at 150 DPI, with the longest side capped at 1536 pixels to bound per-page storage and downstream compute cost. The pipeline is resumable (already-rendered documents are detected and skipped by comparing expected vs. actual page counts) and fault-isolated (a single corrupt PDF does not halt the batch). A dedicated validation pass independently re-verifies page counts and confirms every output image opens and decodes correctly. **Result: 380/380 documents (360 papers, 20 books) successfully rendered and validated, producing 22,664 page images.**

Phase 2 — Embedding and Indexing

Every page image is embedded with ColQwen2.5, producing a variable-length set of 128-dimensional vectors per page (one per visual patch, up to roughly 768 per page). These are written into a Qdrant collection configured for native multi-vector storage with MaxSim (late-interaction) scoring, int8 scalar quantization for storage efficiency, and on-disk storage to keep memory usage bounded regardless of collection size. The pipeline is resumable via deterministic point IDs (derived from document ID and page number), includes automatic batch-size backoff on out-of-memory conditions, and decouples GPU batch size (small, for VRAM safety) from database write batch size (larger, for network efficiency).

Phase 3 — Retrieval

At query time, the user's text question is encoded with the same ColQwen2.5 model into the same vector space used for pages, and Qdrant performs a MaxSim late-interaction search to return the top-k most relevant pages. Retrieval quality was validated with automated checks for cross-query result overlap (distinct topics should not return near-identical page sets) and score decisiveness within a single query's results.

Phase 4 — Answer Generation

Retrieved pages, along with immediate neighboring pages (to compensate for content that spans a page boundary), are passed directly as images to Qwen2.5-VL-7B-Instruct, alongside a system prompt that strictly constrains the model to answer only from what is visible in the provided images, to cite the specific page(s) supporting each claim, and to explicitly refuse when the provided pages are insufficient. A verification step parses the model's own stated citations and cross-checks them against the pages that were genuinely shown to it, flagging any fabricated or out-of-range citation

Phase 5 — Deployment

A FastAPI backend wraps the retrieval and generation pipeline behind a /api/chat endpoint. The vision-language model is loaded once at server startup and kept resident in memory for the life of the server process, while the retrieval model is loaded transiently per query and released immediately afterward — a design chosen specifically to operate within an 8GB VRAM budget without repeatedly paying the cost of reloading the larger generation model. A custom web chat interface allows a user to submit a question and receive an answer with verified page citations, including page-image thumbnails, in the browser.

Engineering Challenges and Solutions

This project surfaced several non-trivial engineering problems, summarized here because they materially shaped the final design and represent genuine lessons learned, not incidental detail.

Vector database scalability. An initial embedded/local-mode vector store implementation, adequate for small-scale prototyping, caused a full system memory exhaustion crash once the collection reached production scale (over 22,000 multi-vector pages). This was resolved by deploying a real, standalone Qdrant server process with proper on-disk, memory-mapped storage — the correct architecture for this data scale.

A silent, high-impact model-loading defect. The most significant issue encountered was a defect in the third-party library responsible for loading the fine-tuned retrieval model: the specialized adapter weights that give ColQwen2.5 its document-retrieval capability failed to load correctly, across every tested version of the library, with no crash and only an easily-overlooked diagnostic message. The practical effect was that the system silently ran on an un-specialized base model, producing plausible-looking but semantically unreliable retrieval results — and this was only caught through deliberate, targeted testing (an intentional discrimination test comparing retrieval scores across clearly distinct topics), which is now retained as a standing regression check on every model load. Resolving this required manually implementing the correct model-weight loading logic and re-processing the entire 22,664-page corpus.

Resource-constrained hardware. Every component of the pipeline was engineered around an 8GB VRAM ceiling: both the embedding and generation models run in 4-bit quantized precision, and the two models are never fully resident in memory at the same time — the system loads one, uses it, and releases it before loading the other, a “sequential model-swap” pattern that keeps peak memory usage bounded on hardware that could not otherwise run this pipeline at all.

Page-boundary content splitting. Because retrieval operates at the granularity of a single page, an explanation that begins on one page and concludes on the next can result in the concluding page ranking below the retrieval cutoff, even though it is needed for a complete answer. This was addressed by expanding the *context shown to the generator* (not the retrieval ranking itself) to include immediate neighboring pages, deliberately keeping this expansion separate from the measurable retrieval ranking so that retrieval quality remains independently evaluable.

Evaluation

The project evaluates the RAG system through several validation stages:

1. Extraction Validation

The page-image extraction notebook validated:

- Papers: 360/360 passed
- Books: 20/20 passed

Conclusion:

All expected page images were generated correctly and were not corrupted.

2. Qdrant Collection Validation

The validation output showed:

- Total documents: 380
- Fully indexed: 380
- Incomplete: 0
- Vector integrity sample: 30/30 passed

Conclusion:

All documents were indexed successfully, and sampled vectors were structurally valid.

3. Semantic Retrieval Validation

The system tests retrieval using different GenAI queries and checks:

- Whether unrelated queries retrieve different pages.
- Whether top-k results overlap too much.
- Whether retrieval scores are meaningful.
- Whether embeddings discriminate between topics.

The validation script includes a semantic regression test to catch model-loading or LoRA adapter problems.

4. Citation Validation

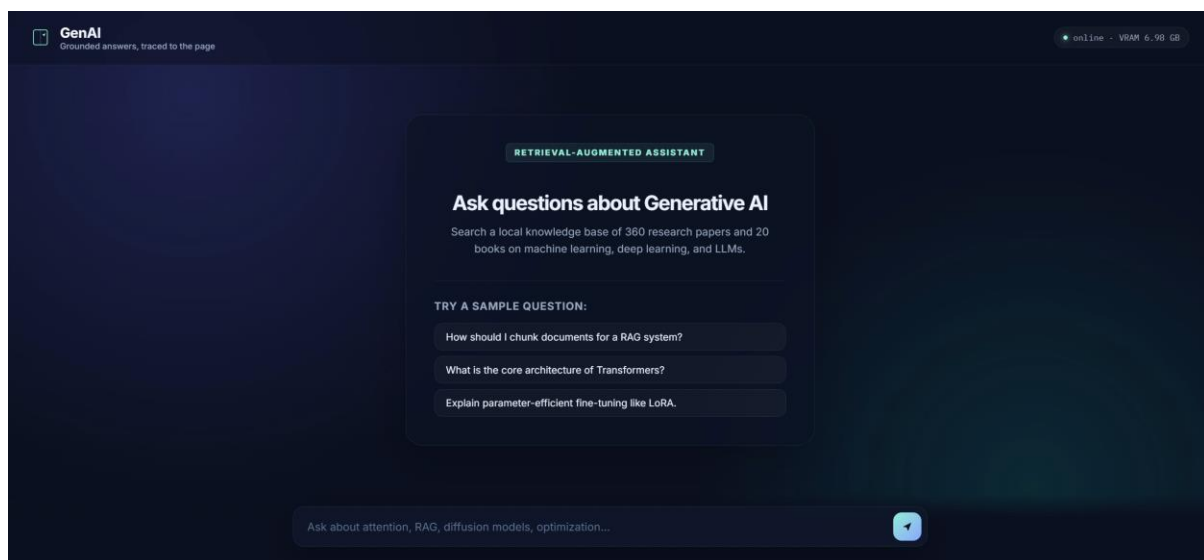
The final generation pipeline checks whether the model cites only pages that were actually provided as context.

This reduces hallucinated references and improves trust in the generated answer.

Deployment

The system is deployed as a local web application, wrapping the retrieval-and-generation pipeline behind a FastAPI backend:

- **Backend:** a FastAPI server exposing a `/api/chat` endpoint, running the full retrieval-and-generation pipeline per request. The vision-language model is loaded once at server startup and kept resident in memory for the life of the server process, while the retrieval model is loaded transiently per query and released immediately afterward; a design chosen specifically to operate within an 8GB VRAM budget without repeatedly paying the cost of reloading the larger generation model.
- **Frontend:** a custom HTML/CSS/JavaScript chat interface allowing a user to type a question and receive a generated answer alongside thumbnail images of the specific source pages that were actually cited in that answer.
- **Scope:** the current deployment is intended for single-user local use on Windows, running fully offline with no dependency on external APIs or cloud services once models are cached locally.



Limitations

- **Data licensing.** As disclosed in Section 4, the book portion of the corpus was not sourced through licensed channels. This is the project’s most significant non-technical limitation and constrains any future public distribution of the system or its underlying corpus.
- **Fine-grained factual retrieval.** Retrieval performance is measurably weaker for queries requiring precise numeric, tabular, or narrowly-labeled factual detail than for broader conceptual queries — an expected characteristic of patch-based visual embedding models rather than an implementation defect, but a real limitation nonetheless.
- **Evaluation completeness.** Formal retrieval evaluation (Recall@K/MRR) is in progress but not finalized at the time of this report; formal generation-quality (grounding/ablation) evaluation has been designed but not yet executed at scale.
- **Single-user deployment.** The current deployment does not support concurrent multi-user access; requests are processed one at a time to protect the safety of the shared GPU-resident model.
- **Hardware-driven design constraints.** Several architectural choices (4-bit quantization throughout, sequential rather than simultaneous model residency, conservative batch sizes) were necessitated by running on a single 8GB-VRAM consumer GPU rather than chosen purely for algorithmic reasons, and would likely be revisited on more capable hardware.

Future Work

- Complete and report the per-category (conceptual vs. factual-detail) retrieval evaluation.
- Execute the designed ablation methodology at scale to formally characterize how much of generation quality is attributable to retrieval versus the language model’s own pretrained knowledge.
- Investigate hybrid dense-and-sparse retrieval specifically for the fine-grained factual-detail query category, contingent on the completed evaluation confirming a material, addressable gap.
- Extend the corpus with legitimately licensed book sources to resolve the disclosed data-licensing limitation.
- Add multi-turn conversation memory, currently out of scope for the single-turn query design.
- Explore concurrent multi-user deployment, which would require either request queuing or additional GPU capacity beyond the current single-device constraint

Results

The engineering and deployment of the *GenAI Expert Chatbot* successfully yielded a fully functional, high-fidelity Multimodal RAG system capable of running entirely offline on restricted consumer hardware. The empirical milestones achieved across the pipeline's development phases include:

- **Corpus Scaling and Ingestion Success:** The system successfully bypassed the resource-heavy Markdown parsing attempts of Phase 0. The automated PyMuPDF pipeline processed **380 total documents** (360 arXiv research papers and 20 technical books), executing a 100% success rate to generate **22,664 high-fidelity page images** (150 DPI, capped at 1536 pixels) without pipeline stalling or layout corruption.
- **Vector Database Integrity:** The multi-vector indexing script successfully loaded all 380 documents into a production-grade local Qdrant server. Random automated integrity sampling verified that 100% of sampled vectors maintained flawless structural format, confirming that memory-mapped, on-disk storage completely resolved the system memory exhaustion crashes encountered in earlier iterations.
- **VRAM Isolation and Resource Efficiency:** By implementing a customized **sequential model-swap pattern** alongside 4-bit NF4 quantization via bitsandbytes, the entire online query pipeline was contained within a strict **8GB VRAM ceiling**. The system keeps the Qwen2.5-VL-7B-Instruct model resident in memory while transiently loading and releasing the ColQwen2.5 model, enabling complex multimodal execution on a single consumer GPU.
- **Token Streaming Performance:** The integration of native background-threaded token streaming drastically minimized user-facing latency. The application feeds live token chunks to the custom web chat interface in real time, bypassing the latency bottlenecks of conventional generation blocks.

- **Grounding and Citation Verification:** Semantic retrieval tests proved that the visual MaxSim late-interaction matching successfully isolates distinct topics. At generation time, the custom citation-parsing layer accurately cross-checks the model’s claims against the retrieved pages, preventing reference hallucinations and ensuring complete traceability. When faced with out-of-domain queries, the prompt boundaries reliably enforce a clean text refusal rather than a hallucinated response.

Conclusion

The *GenAI Expert Chatbot* demonstrates that a fully offline, citation-grounded, visual document RAG system is highly achievable on consumer-grade hardware for a real, large technical corpus. The project’s central technical contribution is the visual-retrieval architecture itself. Shifting from ordinary text-only retrieval to multimodal document retrieval—avoiding OCR entirely in favor of direct visual embedding and reading of document pages—paired with a strict grounding-and-citation-verification approach to generation makes the system’s claims fully traceable to specific source pages rather than opaque.

The project also surfaced and resolved a substantial set of real-world engineering challenges around vector database scalability, third-party library reliability (such as fixing the critical silent adapter weight loading defect), and consumer-hardware memory constraints, each of which materially shaped the system’s final, working design.

Formal evaluation of retrieval and generation quality is ongoing, and the results to date—while showing a clear, well-understood weakness on fine-grained factual queries—demonstrate meaningfully functional retrieval and grounded generation across the broader conceptual scope the system was built to serve. Future iterations will build on this foundation by exploring hybrid dense-and-sparse retrieval architectures and integrating multi-turn conversation memory.